

Cloud Foundry VS Kubernetes

Two very different answers to the same question: how do we get code from a developer's laptop to production, and keep it running there, without every team reinventing the plumbing?



CATEGORY	UNIT OF DEPLOYMENT	YOU WRITE
Platform as a Service Container orchestrator	Application Pod	Source + manifest.yml Image + YAML manifests
GUIDING PRINCIPLE	SWEET SPOT	FIRST RELEASED
Convention over configuration Primitives you compose	Stateless twelve-factor apps Any containerised workload	2011 2014

Contents 1 Overview · 2 Philosophy · 3 Architecture · 4 Developer experience · 5 Feature-by-feature · 6 Operations · 7 Strengths & weaknesses · 8 Choosing · 9 Glossary
Prepared September 2026

1 Overview

Cloud Foundry (CF) is an opinionated *application platform*: you hand it source code and it does the rest. Kubernetes (k8s) is a general-purpose *container orchestrator*: you hand it containers plus a precise description of how they should run, and it makes reality match that description.

APPLICATION PLATFORM · PAAS

Cloud Foundry

- Born at VMware in 2011, later Pivotal; now a Cloud Foundry Foundation project under the Linux Foundation.
- Core promise: `cf push` and your app is built, packaged, scheduled, routed, logged and health-checked.
- Buildpacks turn source into a runnable image; you never write a Dockerfile.
- Strong multi-tenancy model: orgs, spaces, quotas, roles out of the box.
- Modern distributions (Korifi, cf-for-k8s lineage) run *on top of* Kubernetes.

CONTAINER ORCHESTRATOR · CAAS

Kubernetes

- Open-sourced by Google in 2014, based on Borg; graduated CNCF project and de-facto industry standard.
- Core promise: declare desired state in YAML; controllers reconcile the cluster toward it continuously.
- Runs any OCI container: web apps, batch jobs, databases, ML pipelines, network functions.
- Everything is an extensible API object; a vast ecosystem fills the gaps (Helm, Istio, ArgoCD, Knative...).
- Available from every cloud (EKS, GKE, AKS) and on-prem (OpenShift, Rancher, k3s).

The one-sentence summary. Cloud Foundry optimises for the *developer* who wants to ship a twelve-factor app and never think about infrastructure. Kubernetes optimises for the *platform team* who wants a universal substrate on which to build whatever is needed.

2 Philosophy: opinions vs primitives

The deepest difference is not technical but cultural. Cloud Foundry ships with a strong opinion about what an application is: a stateless, horizontally scalable process that speaks HTTP, reads configuration from the environment, and logs to stdout. If your workload fits that shape, CF removes almost every decision from your path. If it does not, you fight the platform.

Kubernetes refuses to have that opinion. It offers primitives (Pods, Services, Volumes, Jobs, ConfigMaps) and lets you compose them. The price is that *somebody* has to hold the opinions: which ingress controller, which secrets store, which CI pipeline, which observability stack. In a mature organisation that somebody is a platform team building an internal developer platform (IDP). In a small team it is often nobody, and the YAML sprawls.

What each platform asks of you (dashed orange boxes are what you author and maintain)

Cloud Foundry

Your source code + <code>manifest.yml</code>	YOU OWN THIS
Buildpacks (detect, compile, release)	PLATFORM
Diego scheduler · Garden containers	PLATFORM
Gorouter · Loggregator · UAA · Cloud Controller	PLATFORM
Service brokers (databases, queues, caches)	PLATFORM
BOSH-managed VMs or a Kubernetes cluster	INFRASTRUCTURE

Kubernetes

Your source code + Dockerfile / Buildpack	YOU OWN THIS
Deployment, Service, Ingress, HPA, ConfigMap, Secret YAML	YOU OWN THIS
Ingress controller, cert-manager, logging, metrics, service mesh	YOU / PLATFORM TEAM
API server · etcd · scheduler · controllers · kubelet	PLATFORM
Operators and CRDs for stateful services	PLATFORM + ECO
Managed cloud service or self-hosted nodes	INFRASTRUCTURE

3 Architecture

Cloud Foundry components

COMPONENT	ROLE	NEAREST KUBERNETES ANALOGUE
Cloud Controller (CAPI)	REST API; stores apps, spaces, orgs, service bindings; drives staging and deployment.	kube-apiserver + etcd
Diego	Scheduler and runtime: BBS state store, auctioneer bids work to cells, reps run it.	kube-scheduler + kubelet
Garden	Container runtime abstraction (runC underneath).	containerd / CRI-O
Gorouter	Built-in HTTP/TCP router; every app gets a route automatically.	Ingress controller + Service (nothing is automatic)
Loggregator	Streams app logs and metrics to <code>cf logs</code> and to drains.	Fluent Bit / Loki / Prometheus (bring your own)
UAA	OAuth2 identity server for users and clients.	OIDC provider + RBAC
Buildpacks	Turn source into a droplet: language detection, dependency install, start command.	Cloud Native Buildpacks / kpack (optional)
Service brokers (OSBAPI)	Provision and bind backing services with one command.	Operators, Crossplane, Service Binding spec
BOSH	Deploys and upgrades the platform itself across VMs.	Cluster API / managed control plane

Kubernetes components

COMPONENT	ROLE	NEAREST CLOUD FOUNDRY ANALOGUE
kube-apiserver	The single front door; every object is created, read and watched here.	Cloud Controller
etcd	Consistent key-value store holding all cluster state.	CAPI database + Diego BBS
kube-scheduler	Assigns Pods to nodes based on resources, affinities and taints.	Diego auctioneer
Controllers	Reconcile loops for Deployments, ReplicaSets, Jobs, Nodes, endpoints...	Diego + Cloud Controller workers
kubelet	Node agent that runs and reports on Pods.	Diego rep
kube-proxy / CNI	Cluster networking and Service virtual IPs.	Silk / CF networking
CRDs + Operators	Extend the API with new resource types and their controllers.	No direct equivalent (CF is closed for extension)

Same shape, different scope. Both are API-driven control planes with a scheduler, node agents and a state store. The difference is that Cloud Foundry bundles the router, log pipeline, identity and build system as first-class parts of the control plane, while Kubernetes leaves them to add-ons.

4 Developer experience

The clearest way to feel the difference is to deploy the same small web app on each.

Cloud Foundry

manifest.yml

```
applications:
- name: hello
  memory: 256M
  instances: 2
  buildpacks:
  - python_buildpack
  services:
  - hello-db
```

Deploy, scale, inspect:

```
$ cf push
$ cf scale hello -i 4
$ cf logs hello
$ cf create-service postgres small hello-db
$ cf bind-service hello hello-db
```

Route, TLS, health check, logs and the credentials in `VCAP_SERVICES` all appear with no further configuration.

Kubernetes

Dockerfile + at least three manifests:

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: hello }
spec:
  replicas: 2
  selector: { matchLabels: { app: hello } }
  template:
    metadata: { labels: { app: hello } }
    spec:
      containers:
      - name: hello
        image: registry/hello:1.4.2
        resources:
          limits: { memory: 256Mi }
        readinessProbe:
          httpGet: { path: /healthz, port: 8080 }
--- plus Service, Ingress, Secret, HPA,
and a database wired in by hand.
```

Deploy, scale, inspect:

```
$ docker build -t registry/hello:1.4.2 . && docker push ...
$ kubectl apply -f k8s/
$ kubectl scale deploy/hello --replicas=4
$ kubectl logs -l app=hello -f
```

Neither approach is wrong. The Kubernetes version is more verbose because it is more explicit: every decision the CF buildpack and router made silently is now visible and changeable. Teams that need that control love it. Teams that just want the app running see it as ceremony. Tools like Helm, Kustomize, Skaffold, Tilt and Knative exist precisely to shrink this gap, at the cost of yet another layer to learn.

Day-to-day workflow comparison

TASK	CLOUD FOUNDRY	KUBERNETES
Build an artefact	Automatic via buildpack during push.	You build and push an image (CI, Docker, Buildpacks, ko, Jib...).
Expose over HTTP	Automatic route on a shared domain; <code>cf map-route</code> for custom.	Service + Ingress (or Gateway API) + ingress controller + DNS.
TLS	Terminated at Gorouter with platform certs.	cert-manager or cloud LB, configured per Ingress.
Environment config	<code>cf set-env</code> , manifest, user-provided services.	ConfigMap, Secret, env vars, mounted volumes.
Attach a database	<code>cf create-service</code> + <code>cf bind-service</code> .	Operator CRD, cloud console or Terraform; wire credentials manually.
Zero-downtime deploy	Rolling deploys built in; blue/green via <code>cf</code> plugins.	Deployment rolling update built in; canary via Argo Rollouts or Flagger.
Debug a running instance	<code>cf ssh</code> , <code>cf logs</code> , <code>cf events</code> .	<code>kubectl exec</code> , <code>logs</code> , <code>describe</code> , <code>debug</code> , ephemeral containers.
Scheduled task	<code>cf run-task</code> (one-off); scheduler is an add-on.	Job and CronJob are core resources.

5 Feature-by-feature comparison

DIMENSION	CLOUD FOUNDRY	KUBERNETES
Abstraction level	Application. The unit is an app with instances.	Container. The unit is a Pod, composed upward into Deployments, StatefulSets, Jobs.
Workload types	Stateless HTTP/TCP apps, one-off tasks. Stateful workloads are unsupported by design.	Anything containerisable: stateless, stateful, batch, daemon, GPU, serverless (via Knative).
Packaging	Source via buildpacks, or a Docker image.	OCI images only. How you build them is your business.
Configuration model	Imperative CLI plus a small manifest. Limited surface, hard to misconfigure.	Declarative YAML for everything. Huge surface, GitOps-friendly, easy to misconfigure.
Extensibility	Buildpacks, service brokers, CLI plugins. The core is not pluggable.	CRDs, operators, admission webhooks, scheduler plugins, CSI/CNI/CRI. Everything is pluggable.
Multi-tenancy	Orgs, spaces, quotas, roles built in and well understood.	Namespaces, RBAC, ResourceQuota, NetworkPolicy. Workable but assembled by hand; hard multi-tenancy usually means separate clusters.
Networking	Gorouter plus container-to-container policies. Simple and sufficient for most web apps.	CNI plugin of your choice, Services, Ingress/Gateway, NetworkPolicy, optional service mesh. Powerful and complicated.
Storage	Ephemeral disk only; volume services exist but are a niche.	PersistentVolumes, StorageClasses, CSI drivers, snapshots. First-class.
Autoscaling	App Autoscaler add-on: CPU, memory, throughput, schedule based.	HPA, VPA, KEDA (event driven), Cluster Autoscaler / Karpenter for nodes.
Health & self-healing	Port, HTTP or process health checks; crashed instances restarted automatically.	Liveness, readiness and startup probes; restart policies; PodDisruptionBudgets.
Observability	Loggregator, Firehose, log drains, app metrics. Integrated but less flexible.	Nothing built in beyond <code>kubectl logs</code> ; Prometheus, OpenTelemetry, Loki etc. are the norm.
Security posture	Non-root containers, rootfs from stemcells, automatic CVE patching via buildpack and stack updates.	Pod Security Standards, RBAC, secrets, image scanning, policy engines (Kyverno, OPA). Secure when configured; insecure by default in many installs.
Ecosystem & hiring	Mature but shrinking; specialised skill set.	Enormous and growing; the most common skill on platform-engineering CVs.
Vendor landscape	VMware Tanzu (Broadcom), SAP BTP, Cloud.gov, open-source cf-deployment and Korifi.	Every hyperscaler, OpenShift, Rancher, Tanzu, k3s, Talos, and dozens more.
Runs on	BOSH-managed VMs on any IaaS, or Kubernetes (Korifi).	Bare metal, VMs, every cloud, edge devices, laptops.

6 Operating the platform

Developer experience is where Cloud Foundry shines. Operations is where the trade-offs get interesting.

Installation and upgrades

Classic Cloud Foundry is deployed with BOSH, which manages the full VM lifecycle including OS images (stemcells). Upgrades are a well-rehearsed, mostly rolling process, but BOSH itself is a large, unfamiliar tool with a small community. A full CF foundation is heavy: dozens of VMs before the first app is pushed.

Kubernetes is available as a managed control plane from every cloud, which removes most of the installation burden. On-prem, tools like kubeadm, Cluster API, Rancher, Talos and OpenShift cover it. Upgrades are frequent (three minor releases a year, each supported for roughly fourteen months), and each one can break API versions or add-ons, so operators spend real effort staying current.

Day-2 concerns

CONCERN	CLOUD FOUNDRY	KUBERNETES
Patching CVEs in app runtimes	Update the buildpack or stack, restage apps. Platform-wide and fast.	Rebuild every image, redeploy. Requires CI discipline and image scanning.
Capacity management	Add Diego cells; the auctioneer balances load.	Node pools, autoscalers, requests/limits tuning, bin-packing analysis.
Backups and DR	BBR (BOSH Backup and Restore) for platform state.	etcd snapshots plus Velero or similar for workloads and volumes.
Cost visibility	Per-org and per-space quotas; simple chargeback.	Namespace quotas plus tools such as OpenCost or Kubecost.
Team size to run it	Small platform team once stood up; steep learning curve to start.	Small if fully managed; substantial if self-hosted with a full add-on stack.
Blast radius of mistakes	Limited: developers cannot touch the platform layer.	Wide: a bad RBAC rule or admission webhook can take down a cluster.

The convergence story. The CF community responded to Kubernetes rather than fighting it. *Korifi* reimplements the Cloud Foundry API as Kubernetes CRDs and controllers, using Cloud Native Buildpacks (kpack) and Contour, so `cf push` works against an ordinary cluster. Cloud Native Buildpacks themselves are a CNCF project born from CF and Heroku buildpacks. Meanwhile projects such as Knative, Backstage and various internal developer platforms rebuild the "just push code" experience on Kubernetes from the other direction. The two worlds are meeting in the middle.

7 Strengths and weaknesses

Cloud Foundry

STRENGTHS

- Fastest path from source to running, routed, logged application.
- Developers need to learn one command and one small manifest.
- Strong guardrails: consistent security, no root, patched runtimes, quotas.
- Multi-tenancy and RBAC that match how enterprises are organised.
- Service marketplace makes backing services a one-liner.
- Proven at scale in regulated industries (banks, insurers, government).

WEAKNESSES

- Opinionated: stateful, non-HTTP or unusual workloads fit badly or not at all.
- Heavy footprint and BOSH learning curve for the classic distribution.
- Shrinking community and vendor pool; hiring is harder every year.
- Limited extensibility; if the platform lacks a feature you wait for it.
- Less momentum in tooling and integrations than the CNCF ecosystem.
- Perceived as legacy, which affects buy-in regardless of technical merit.

Kubernetes

STRENGTHS

- Runs every kind of workload on every kind of infrastructure.
- Declarative API is a natural fit for GitOps and infrastructure as code.
- Unmatched ecosystem: meshes, operators, observability, policy, ML, serverless.
- Managed offerings from every cloud reduce the operations burden.
- Largest talent pool and knowledge base in the industry.
- Extensible to the core; you are never blocked waiting for upstream.

WEAKNESSES

- Steep learning curve; developers must understand infrastructure concepts.
- Verbose configuration; an app needs many YAML objects to be production-ready.
- No batteries included: ingress, logging, secrets, CI must all be chosen and assembled.
- Frequent releases and deprecations create ongoing upgrade toil.
- Easy to build an insecure or unreliable cluster; many defaults are permissive.
- Complexity tends to leak upward to application teams unless a platform team absorbs it.

8 Choosing between them

The honest answer in 2026 is that most new platforms are built on Kubernetes, and the real question is *which developer experience you put on top of it*. Cloud Foundry remains an excellent answer where it is already installed, where the workload shape fits, and where the organisation values guardrails over flexibility.

Lean towards Cloud Foundry when...

- Your fleet is mostly stateless web services and APIs.
- You want developers to never see a Dockerfile or YAML.
- Compliance requires strong, uniform guardrails across hundreds of teams.
- You already run CF and it is meeting your needs.
- A single vendor with a supported, integrated product is preferred.
- You can run Korifi and get both CF ergonomics and a Kubernetes substrate.

Lean towards Kubernetes when...

- Workloads include stateful services, batch, ML, GPUs or anything unusual.
- You want one substrate for everything, including third-party software.
- GitOps and declarative infrastructure are part of your culture.
- You have (or will build) a platform team to curate the stack.
- Hiring, community and long-term ecosystem momentum matter.
- You are starting fresh with no existing platform investment.

A pragmatic middle path

Treat Kubernetes as the operating system and buy or build the application platform on top. Options in rising order of effort: a managed PaaS-on-k8s (OpenShift, Tanzu Application Platform), an open-source layer (Korifi, Knative plus Cloud Native Buildpacks), or a bespoke internal developer platform stitched from Backstage, ArgoCD, Crossplane and Helm. Whatever you pick, the goal is the same one Cloud Foundry set in 2011: a developer should be able to push code and walk away.

9 Glossary

BOSH

Cloud Foundry's tool for deploying and managing the platform's own VMs across any IaaS.

Buildpack

A script set that detects an app's language, installs dependencies and produces a runnable image. Now standardised as Cloud Native Buildpacks.

CNCF

Cloud Native Computing Foundation, home of Kubernetes, Prometheus, Envoy, Buildpacks and many more.

CRD

Custom Resource Definition; the mechanism for adding new object types to the Kubernetes API.

Diego

Cloud Foundry's container scheduler and runtime layer.

Droplet

The staged, ready-to-run artefact a buildpack produces from source in Cloud Foundry.

Gorouter

Cloud Foundry's built-in HTTP router that maps routes to app instances.

HPA

Horizontal Pod Autoscaler; scales Kubernetes replicas on metrics.

IDP

Internal Developer Platform; an organisation's curated, self-service layer over raw infrastructure.

Ingress

Kubernetes object describing external HTTP routing; requires a controller to act on it.

Korifi

Cloud Foundry's Kubernetes-native reimplementation, exposing the CF API as CRDs.

Loggregator

Cloud Foundry's log and metric streaming subsystem.

OCI image

The standard container image format consumed by Docker, containerd, Kubernetes and Cloud Foundry alike.

Operator

A Kubernetes controller that manages a complex application (often stateful) using CRDs.

Org / Space

Cloud Foundry's two-level tenancy hierarchy for grouping apps, users and quotas.

Pod

The smallest deployable unit in Kubernetes: one or more containers sharing network and storage.

Service broker

An Open Service Broker API implementation that provisions and binds backing services in CF.

Twelve-factor app

A methodology for building portable, stateless, config-from-environment services; the workload shape CF was designed for.

UAA

User Account and Authentication; Cloud Foundry's OAuth2 identity server.